

LE FORMAT PARQUET POUR LA DIFFUSION DE DONNÉES : UN CHOIX TECHNIQUE AU SERVICE DES UTILISATEURS

Cédric Bobinec (), Lino Galiana (**)*

() Insee, Pôle Didoc*

*(**) Insee, Direction des statistiques démographiques et sociales*

cedric.bobinec@insee.fr lino.galiana@insee.fr

Mots-clés : Open source, open data, diffusion de données

Domaines : Open data, documentation, métadonnées, mise à disposition, Communication, open science

Résumé

La diffusion et l'exploitation de microdonnées se heurte souvent à des freins techniques liés au volume des fichiers et à la complexité de leur exploitation. Le format Parquet, issu à l'origine de l'écosystème big data mais s'étant diffusé bien au-delà de cette sphère, s'impose comme une alternative efficace aux formats traditionnels comme le CSV. Cette présentation reviendra sur les expériences de diffusion de données dans ce format. Ce choix technique, associé à un accompagnement des utilisateurs (guides, exemples de code), a permis d'accroître l'usage de ces sources qui étaient, auparavant, complexes à valoriser. Cette communication évoquera également les implications pratiques de ce choix technique, les bonnes pratiques à mettre en oeuvre lors de l'adoption de ce format de données et les opportunités d'innovation permises par ce format.

Abstract

The dissemination and use of microdata often face technical obstacles related to the size of the files and the complexity of working with them. The Parquet format, originally developed within the big data ecosystem, is now widely adopted. It has emerged as an efficient alternative to traditional formats such as CSV. This presentation will discuss experiences in distributing data in this format. The technical choice of Parquet, combined with user support (guides, code examples), has helped increase the use of these data sources, which were previously difficult to exploit. The talk will also address the practical implications of this technical choice, best practices for adopting this data format, and the opportunities for innovation that it enables.

1. Introduction

Parmi les nombreuses innovations techniques ayant émergé au cours des années 2010 dans l'écosystème de la donnée, le format Parquet fait parti de celles ayant connues une adoption large parmi les *data scientists* et statisticiens. Initialement conçu pour répondre aux besoins des plateformes *big data* comme [Hadoop](#) - réduction du volume de stockage et du temps de lecture des données - celui-ci s'est diffusé au-delà de cette sphère et a même survécu à la plupart des technologies *big data* qui sont, depuis, tombées en désuétude (Tigani, [2023](#)).

Plusieurs acteurs majeurs de l'écosystème de l'intelligence artificielle ont fait de Parquet la pierre angulaire de leur lac de données. Dans le domaine de la diffusion de données, où se rencontrent des publics aux attentes et compétences techniques diverses, l'adoption de ce format a été plus tardive. Si la plateforme communautaire `data.gouv` recense depuis plusieurs années des Parquet créés par des utilisateurs experts pour faciliter l'usage de certains jeux de données, la statistique publique ne s'est mise à diffuser, d'elle-même, que récemment des fichiers Parquet. Suite à la publication en 2023 des données détaillées du répertoire électoral unique (REU) puis du recensement de la population (RP), ce format est devenu, du fait de l'accueil positif qu'il a reçu, le moyen privilégié de diffuser des données volumineuses en *open data* pour la statistique publique.

Cet article vise à offrir un complément au retour d'expérience présenté lors des Journées de méthodologie statistique (JMS) 2025. Par rapport à celui-ci, cet article présentera plus extensivement les enjeux techniques qui expliquent la popularité de ce format bien qu'elles ne soient pas propres aux problématiques de diffusion de données (Section 2). Cela permettra de justifier la manière dont ce choix technique rend possible de nombreuses exploitations. Nous reviendrons ensuite sur les enjeux liés à la diffusion de données publiques en Parquet et proposerons quelques bonnes pratiques pour leur mise à disposition et l'accompagnement des utilisateurs (Section 3). Enfin, nous évoquerons les perspectives qu'ouvre une adoption plus large de ce format pour la diffusion et le partage de données (Section 4).

2. Retour sur les propriétés techniques du format Parquet

2.1. Les deux approches : le fichier ou la base de données

Pour bien comprendre la manière dont Parquet, ou des variantes de celui-ci, s'est imposé comme le format de prédilection des acteurs de l'écosystème de la donnée, il est nécessaire de revenir sur les enjeux techniques de celui-ci. Pour remettre en perspective cette solution technique, il convient de revenir sur la dualité existant dans le domaine du stockage et de la mise à disposition de données entre les bases de données ou les fichiers. Ces deux logiques diffèrent fortement dans la manière dont l'information est organisée, accessible et modifiable par un client qui la consomme.

Les bases de données relationnelles

Les bases de données relationnelles servent, au moins depuis les années 1970, dans de nombreux entrepôts de données à organiser l'information lorsqu'elle prend une forme tabulaire¹. Il s'agit d'une logique systémique. L'accès à la donnée (en lecture comme en écriture) est gérée par un système de gestion de base de données (SGBD) qui permet :

- de stocker des ensembles cohérents de données,
- d'en permettre la mise à jour (ajout, suppression, modification),
- d'en contrôler l'accès (droits utilisateurs, types de requêtes, etc.).

Dans ces systèmes, les données sont organisées en tables reliées par des relations, suivant généralement un schéma en étoile. Par exemple, une table longitudinale de données individuelles est reliée d'un côté à une table d'identifiants ménage uniques et de l'autre à une table d'identifiants entreprise unique.

Le logiciel associé à la base de données fait ensuite le lien entre ces tables, souvent par le biais de requêtes SQL. L'un des logiciels les plus efficaces dans ce domaine est [PostgreSQL](#).

Les fichiers

¹La diversification des données produites, notamment le fait que de nouveaux objets puissent être considérés comme de la donnée (texte, image, vidéo...), fait que d'autres approches de bases de données se sont développées pour les données non structurées où la nature coercitive du schéma en étoile n'était pas idéal. On parle pour désigner ce système de *NoSQL* (« *not only SQL* »). Comme les enjeux liés à ce type de source sont particuliers, et qu'ils concernent peu la diffusion de données de statistique publique, qui par essence est principalement tabulaire, nous n'évoquerons pas ces enjeux.

La deuxième approche est celle du fichier qui consiste à organiser l'information dans un document stocké de manière autonome sur un système de fichiers. Contrairement à la base de données, le fichier ne repose pas sur un moteur logiciel chargé d'en gérer les accès ou la cohérence : il contient directement la donnée, dans un format plus ou moins structuré selon les besoins.

Cette approche favorise la simplicité et la portabilité. Les fichiers peuvent être partagés, copiés ou archivés sans dépendre d'une infrastructure spécifique. Ils ne nécessitent pas l'installation ou le maintien d'un logiciel de gestion spécialisé comme PostgreSQL : un simple *file system*, déjà présent sur tout système d'exploitation, suffit à y accéder. Leur lecture ne nécessite qu'un outil adapté au format utilisé.

Le succès croissant des fichiers dans l'écosystème de la data science s'explique par plusieurs facteurs techniques et pratiques qui les rendent particulièrement adaptés aux usages analytiques modernes.

En premier lieu, les fichiers sont beaucoup plus légers à manipuler que les bases de données. Ils ne nécessitent pas l'installation ou le maintien d'un logiciel de gestion spécialisé : un simple *file system*, déjà présent sur tout système d'exploitation, suffit à y accéder. En ce qui concerne la lecture, il s'agit d'avoir un outil adapté au format utilisé. Ceci rend l'approche par les fichiers particulièrement souple et explique le succès de formats plats comme le CSV (lisible par n'importe quel éditeur de texte), JSON (lisible par n'importe quel navigateur web) ou, plus récemment, le format Parquet.

La principale raison pour laquelle les fichiers sont préférables aux SGBD pour les statisticiens et *data scientists* réside dans la nature des opérations qu'ils effectuent. Les bases de données relationnelles prennent tout leur sens lorsqu'il s'agit de gérer des écritures fréquentes ou des mises à jour complexes sur des ensembles de données structurés², c'est-à-dire dans une logique applicative où la donnée évolue continuellement (ajout, modification, suppression). À l'inverse, dans un contexte analytique, on se contente généralement de lire et de manipuler temporairement des données sans modifier la source. L'objectif est d'interroger, d'agréger, de filtrer pour une valorisation annexe - pas de pérenniser des changements sur une donnée brute. Pour ce type d'usage, les fichiers, notamment dans des formats optimisés comme Parquet, sont parfaitement adaptés : ils offrent une lecture rapide, une portabilité élevée et n'imposent pas l'intermédiation d'un moteur de base de données.

2.2. Les limites du CSV

Si les fichiers sont plus simples d'usage pour les besoins analytiques que les bases de données, tous les formats ne se valent pas. Un format est une manière de contraindre l'organisation de la donnée ce qui conditionnera ultérieurement les possibilités d'exploitation, de stockage et d'interopérabilité.

L'interopérabilité, c'est-à-dire la capacité à ingérer la donnée sans imposer un logiciel, est une propriété désirable d'un fichier de donnée. A cet égard, les formats propriétaires, par exemple le *.sas7bdat* ou le *.xlsx* peuvent poser problème. Bien qu'il existe des bibliothèques dans les logiciels statistiques *open source* pour lire ce type de fichier, elles ne sont pas forcément optimisées et l'intégrité de la donnée n'est pas garantie même si, en général, l'importation ne pose pas de problème.

Les formats ouverts et documentés garantissent la possibilité pour tout utilisateur, quel que soit son environnement logiciel, de lire et de traiter les données sans dépendre d'un éditeur particulier. Du fait de sa simplicité, le format CSV s'est imposé comme un standard de partage de données. Son principal avantage est qu'il est à la fois lisible par un humain (un simple éditeur de texte permettant de l'ouvrir) et par des programmes informatiques (du fait de sa structure tabulaire).

Cependant, ce format, bien qu'ouvert et universellement lisible, présente de nombreuses limites techniques. En premier lieu, le CSV n'est pas un format compressé donc les fichiers peuvent rapidement devenir volumineux. Ce

²Les bases de données ont une propriété dite ACID (Atomicité, Cohérence, Isolation et Durabilité) qui garantit que chaque transaction est exécutée entièrement ou pas du tout, qu'elle préserve l'intégrité des données, qu'elle ne perturbe pas les autres opérations simultanées et qu'elle reste enregistrée même en cas de défaillance du système.

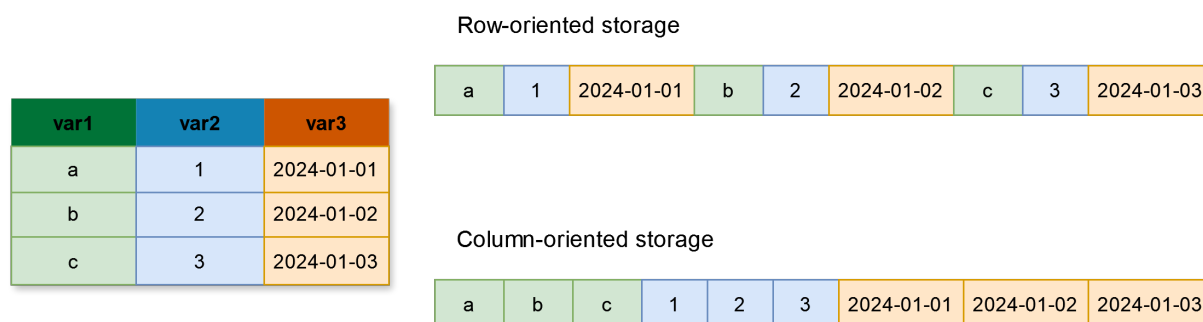


Figure 1. – Organisation de la donnée dans différents formats (empruntée à Avouac et Galiana (2025))

format n'embarque pas non plus de métadonnées pouvant guider la lecture du fichier, en particulier les types de variables.

Ces limites expliquent l'émergence de formats plus modernes, capables de combiner ouverture, efficacité et richesse structurelle — au premier rang desquels le format Parquet.

2.3. Le format Parquet

Sa principale caractéristique est d'être un format orienté colonne. Cela signifie que, contrairement au CSV, les données de chaque colonne sont stockées dans des blocs séparés Figure 1. Cela permet :

- de charger uniquement les colonnes utiles à une analyse ;
- de compresser plus efficacement les données ;
- d'accélérer significativement les requêtes sélectives.

L'organisation en colonnes du format Parquet permet, à partir des métadonnées décrivant la taille et la structure des blocs (ou *row groups*), de sélectionner directement les colonnes utiles à l'analyse et d'ignorer les segments non pertinents. Cette indexation interne optimise la lecture sélective et limite les accès disque.

Une autre propriété importante du format Parquet est sa capacité native à produire des fichiers partitionnés, c'est à dire à distribuer un fichier de données selon une ou plusieurs clés de partitionnement. Dans la majorité des cas, les traitements statistiques ne concernent pas l'ensemble des données d'une source de données : on voudra souvent restreindre les calculs à une zone géographique, une fenêtre temporelle, etc. Si les données à traiter sont disponibles sous la forme d'un unique fichier, il faudra généralement charger l'ensemble des données en mémoire — a minima, les colonnes pertinentes — pour réaliser les traitements. A contrario, Parquet offre la possibilité native de partitionner un jeu de données selon une variable de filtrage fréquente (à la manière d'un index dans une base SQL) ce qui permet d'optimiser les traitements et d'accroître encore leur efficacité (Figure 2). Dans cet exemple, effectuer une requête faisant intervenir seulement un ou quelques mois de données serait très efficient dans la mesure où seules les partitions pertinentes auront besoin d'être chargées en mémoire. Là encore, cette propriété rend le format Parquet particulièrement pertinent pour les traitements analytiques qui caractérisent les applications de *data science* (Dondon et Lamarche, 2023).

Ces différentes propriétés techniques (compression, rapidité en lecture, typage...) font que le format Parquet n'est pas réservé aux architectures *big data* ou aux données volumineuses. Toute personne prête à utiliser un langage de programmation statistique *open source* comme R ou Python et ayant une utilisation intensive de données pourra bénéficier de ses qualités. Pour profiter pleinement de ses propriétés, il est recommandé de privilégier certaines librairies *open source* très bien intégrées à R ou Python.

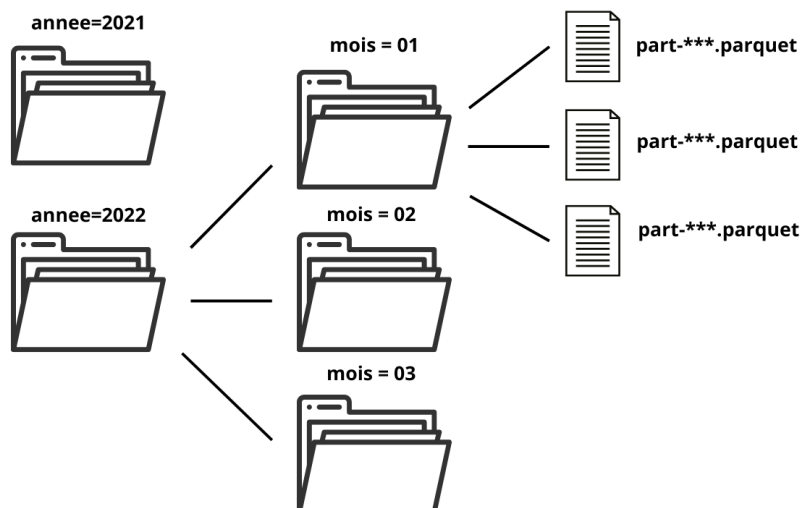


Figure 2. – Représentation sur le *filesystem* d'un fichier Parquet partitionné selon deux clés : l'année et le mois (empruntée à Avouac et Galiana (2025))

2.4. Les bibliothèques pour lire efficacement des Parquet

Le format Parquet rend le stockage de données au format tabulaire beaucoup plus efficient. Mais pour pleinement bénéficier de cette structure de données, il est également nécessaire de s'intéresser à l'étape suivante : le traitement des données en mémoire.

Deux outils majeurs ont émergé à cette fin au cours des dernières années. Le premier est [Apache Arrow](#), un format tabulaire de données en mémoire interopérable entre de nombreux langages (Python, R, Java, etc.). Le second est [DuckDB](#), un système de base de données portable et interopérable permettant de requêter des sources de données de nature très variée (Figure 4). Ces deux outils, bien que techniquement très différents en termes d'implémentation, présentent des avantages et des gains de performance semblables. D'abord, ils sont tous deux orientés-colonne et travaillent ainsi en synergie avec le format Parquet, dans la mesure où ils font persister les bénéfices de ce format de stockage dans la mémoire (Figure 3).

Ces deux bibliothèques, comme l'illustre la Figure 3, font perdurer en mémoire les avantages du format Parquet pour le stockage. Elles permettent également d'exploiter les avancées des processeurs récents en matière de vectorisation des opérations. Dans l'exemple de la Figure 3, la représentation en colonne des données dans la mémoire permet à la requête de filtrage sur les données de la colonne `session_id` d'être beaucoup plus efficiente que dans un format en mémoire traditionnel. Cette approche optimisée permet d'ailleurs de lire des données de grande dimension, qui peuvent excéder largement la mémoire vive de l'ordinateur ou le serveur consommant la donnée (Mühleisen, 2025).

Par ailleurs, Arrow comme DuckDB permettent tous deux d'augmenter considérablement les performances des requêtes sur les données grâce à l'utilisation de la *lazy evaluation* (« évaluation paresseuse »). Là où les opérations sur des données sont généralement exécutées de manière linéaire par les langages de programmation - par exemple, sélectionner des colonnes et/ou filtrer des lignes, puis calculer de nouvelles colonnes, puis effectuer des agrégations, etc. - Arrow et DuckDB exécutent quant à eux ces dernières selon un plan d'exécution pré-calculé qui optimise de manière globale la chaîne de traitements. Dans ce paradigme, les calculs sont non seulement beaucoup plus performants, mais également beaucoup plus efficaces dans la mesure où ils n'impliquent de récupérer que les données effectivement nécessaires pour les traitements demandés. Ces innovations permettent ainsi d'envisager des traitements basés sur des données dont le volume total dépasse la mémoire RAM effectivement disponible sur une machine.

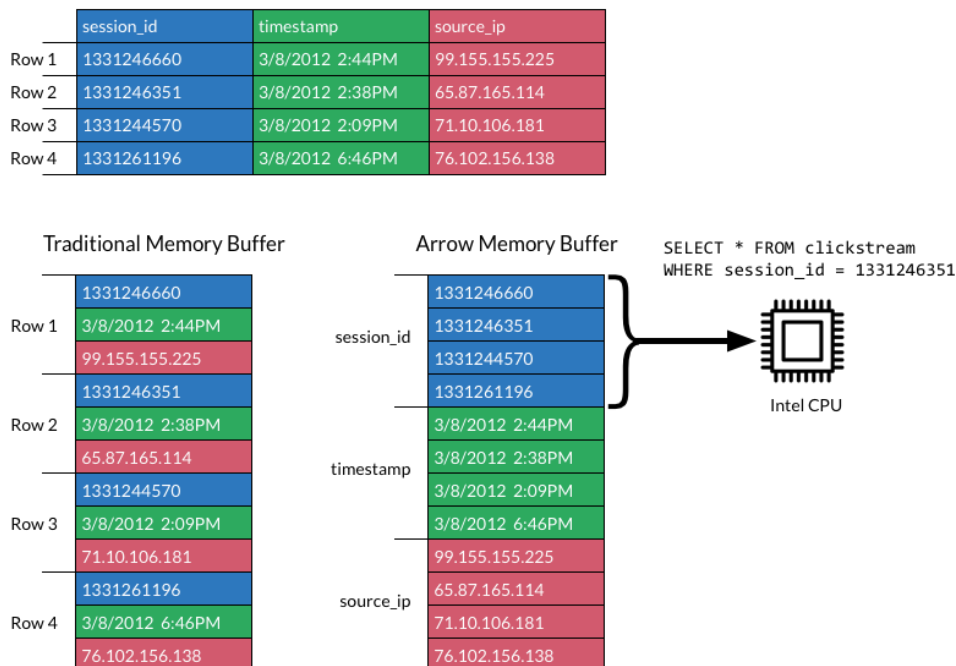


Figure 3. – Représentation en mémoire des données au format Arrow (empruntée à la [documentation du projet Arrow](#))

Un avantage majeur de DuckDB est sa capacité à requêter de manière standardisée des sources de données très variées. DuckDB étant un format de base de données en mémoire, il est naturellement très adapté au requêtage de bases de données relationnelles (comme PostgreSQL ou MySQL). Mais ce *framework* peut également requêter de la même manière des fichiers de données (CSV, Parquet, etc.), qu'ils soient locaux ou stockés dans le *cloud*. Ceci peut être illustré avec quelques exemples progressifs.

Imaginons qu'on s'intéresse au répertoire Sirene et qu'on n'ait besoin que des variables `siret`, `activitePrincipaleEtablissement` et qu'on ne souhaite conserver que les établissements actifs (`etatAdministratifEtablissement` à la modalité A).

Si les données sont stockées au format CSV en local et qu'on utilise DuckDB via Python, il suffit de faire:

Importer un CSV avec DuckDB et Python

```
import duckdb
df = duckdb.sql("""
    FROM read_csv_auto('chemin_local/stock.csv')
    SELECT siret, activitePrincipaleEtablissement
    WHERE etatAdministratifEtablissement == 'A'
""")
df.to_df()
```

Si on bascule sur un format Parquet, le code est presque inchangé:

Importer un Parquet avec DuckDB et Python

```
import duckdb
df = duckdb.sql("""
```

```

FROM read_parquet('chemin_local/stock.parquet')
SELECT siret, activitePrincipaleEtablissement
WHERE etatAdministratifEtablissement == 'A'
"""
).to_df()

```

Maintenant, si le fichier se trouve en ligne, par exemple sur la plateforme *data.gouv*, le code de lecture est presque inchangé. Il est seulement nécessaire d'ajouter l'extension `httpfs` pour pouvoir lire des données non stockées en local:

Importer un Parquet depuis internet avec DuckDB et Python

```

import duckdb

url = "https://object.files.data.gouv.fr/data-pipeline-open/siren/stock/StockEtablissementHistorique_utf8.parquet"
con = duckdb.connect()
con.sql("LOAD httpfs")

df = con.sql(f"""
    FROM read_parquet('{url}')
    SELECT siret, activitePrincipaleEtablissement
    WHERE etatAdministratifEtablissement == 'A'
""")
).to_df()

```

Si finalement le fichier Parquet se trouve sur un système de stockage S3, par exemple sur le SSPCloud, le code change très peu, il suffit que le chemin de lecture commence par `s3://` après avoir créé un paramètre `s3_url_style` en amont:

Importer un Parquet depuis S3 avec DuckDB et Python

```

import duckdb

path = "s3://bucket/sirene.parquet" # Chemin commence par s3://
con = duckdb.connect()
con.sql("LOAD httpfs; SET s3_url_style = 'path';")

df = con.sql(f"""
    FROM read_parquet('{path}')
    SELECT siret, activitePrincipaleEtablissement
    WHERE etatAdministratifEtablissement == 'A'
""")
).to_df()

```

Puisque la lecture et le traitement se font par le biais d'une requête SQL, il est tout à fait possible de directement récupérer un agrégat. Par exemple pour dénombrer les entreprises actives pour chaque APET:

Traiter directement un Parquet depuis S3 avec DuckDB et Python

```

import duckdb

```

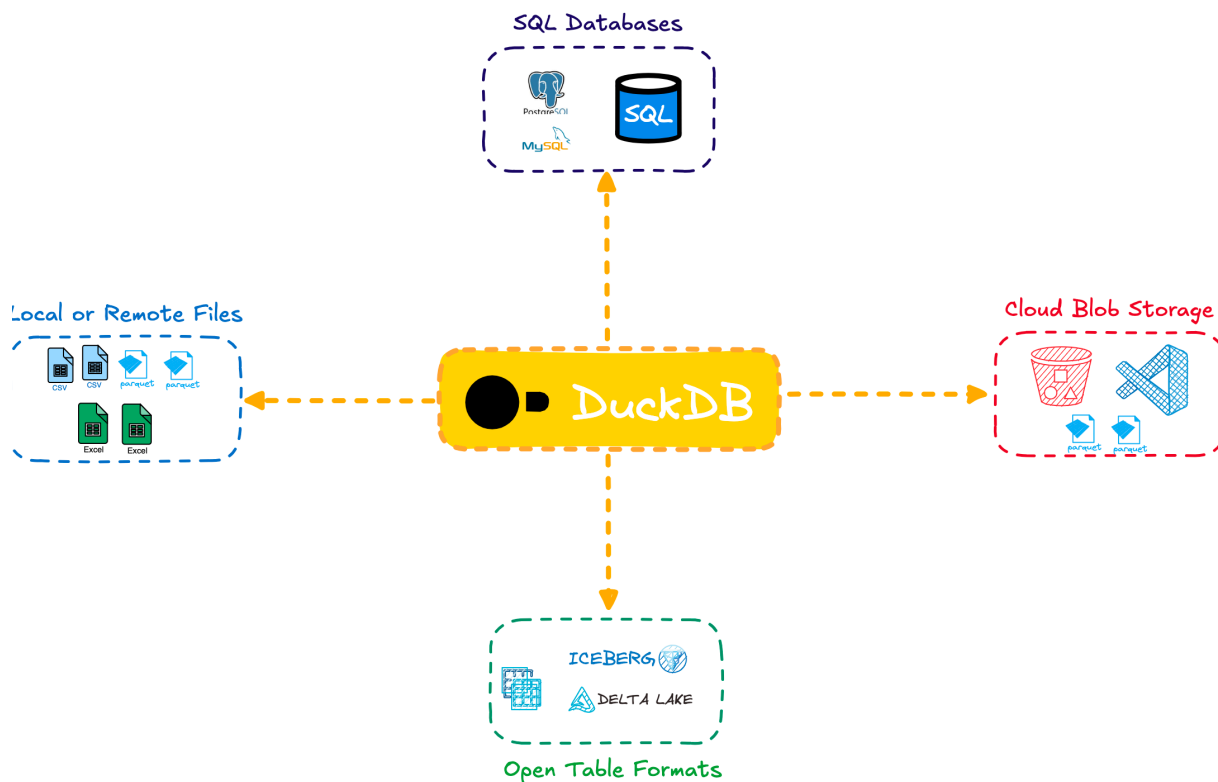


Figure 4. – DuckDB, un outil universel pour lire des données (Source: [MotherDuck](#))

```
path = "s3://bucket/sirene.parquet" # Chemin commence par s3://
con = duckdb.connect()
con.sql("LOAD https://; SET s3_url_style = 'path';")

df = con.sql(f"""
    FROM read_parquet('{path}')
    SELECT count(siret)
    WHERE etatAdministratifEtablissement == 'A'
    GROUP BY activitePrincipaleEtablissement
""")
df.to_df()
```

3. La diffusion de données au format Parquet

Les premières expérimentations de diffusion de données au format Parquet représentaient des cas d'usage idéaux pour tester la réception de ce format par les utilisateurs. Le fichier de diffusion des données détaillées du recensement de la population représente, par exemple, environ 20 millions d'observations pour 80 variables. Ce fichier est à destination d'utilisateurs experts qui désirent construire des statistiques à façon à partir de croisements qui n'existent pas dans les données agrégées.

Ce fichier était historiquement diffusé aux formats dBase (un SGBD dont les origines remontent aux années 1980) et CSV. Un millésime approchait, au format CSV, les 5Go sur disque. Il nécessitait donc des ressources mémoire conséquentes pour être lu dans un logiciel statistique comme R et Python puisque ce format ne permet pas une lecture parcimonieuse du fichier.

Configuration			Taille sur disque (MiB ou GiB)	Vitesse à l'import (secondes)
Format du fichier	Partitionné?	Colonnes		
Ensemble des données (19 735 576 observations)				
CSV		Toutes (88 colonnes)	4.37G	245.59
Parquet		Toutes (88 colonnes)	507.88M	1.44
Parquet		Sous-ensemble (9 colonnes)	507.88M	0.66
Seulement le Centre Val de Loire (683 144 observations)				
CSV		Toutes (88 colonnes)	154.32M	12.71
Parquet	×	Toutes (88 colonnes)	17.23M	0.10
Parquet	✓	Toutes (88 colonnes)	507.88M [†]	1.76
CSV		Sous-ensemble (9 colonnes)	154.32M	11.23
Parquet	×	Sous-ensemble (9 colonnes)	17.23M	0.11
Parquet	✓	Sous-ensemble (9 colonnes)	507.88M [†]	0.52

[†] Poids de l'ensemble des données, y compris régions différentes

Figure 5. – Exemple de comparaison des performances du format Parquet dans plusieurs cas d'usage sur les données détaillées du recensement de la population diffusées par l'Insee en 2023

La comparaison des performances en lecture de ces données au format CSV et Parquet est sans appel³. La mise à disposition d'un fichier n'oblige plus l'utilisateur à faire preuve d'ingéniosité pour découper son fichier de sorte à pouvoir le lire par bloc: cette propriété native du fichier Parquet permet aux utilisateurs de se concentrer sur l'exploitation des données. Les utilisateurs du fichier ne sont généralement pas intéressés par l'intégralité des colonnes: l'orientation colonne de Parquet permettant de n'importer que les variables utiles à l'analyse représente un gain important de confort pour ces utilisateurs. Pour ceux ne s'intéressant qu'à des sous-ensemble de données, par exemple pour des études locales, les retours d'expérience, notamment ceux de Mauviere (2024), ont permis de prendre conscience du rôle important du tri des variables pour l'optimisation des filtres.

Pour aider les utilisateurs à s'approprier ce format, un guide d'utilisation des données a été publié sur le blog du réseau des *data scientists* de la statistique publique, le [SSPHub \(ssphub.netlify.app/\)](https://ssphub.netlify.app/), pour accompagner la diffusion.

Outre une utilisation efficace des ressources, la diffusion dans ce format apporte aussi du confort aux utilisateurs, notamment en montrant comment lire ces données avec DuckDB. Comme cette librairie est simple d'usage, le public cible de la diffusion de ces données, majoritairement utilisateur de R ou Python, pouvait ainsi s'approprier rapidement ce nouveau format. Pour illustrer cette simplicité d'usage, prenons pour exemple un besoin précis: dénombrer les logements niçois dont les occupants ont emménagé depuis 2020. En Python, cela se traduit par la requête SQL:

Un exemple d'utilisation des données diffusées par l'Insee avec DuckDB et Python

```
import duckdb

url = "https://www.data.gouv.fr/api/1/datasets/r/fdf5b008-054b-4065-a819-3eb3313f8be7"
duckdb.sql(
    f"FROM read_parquet('{url}')"
    "SELECT COUNT(*) "
    "WHERE COMMUNE = '06088' and CAST(AEMM AS int) > 2020"
```

³Ce tableau est produit dans le cadre de la formation aux bonnes pratiques R et Git développée par l'Insee et faisant la promotion du format Parquet pour le stockage de données. Pour reproduire celui-ci, se rendre sur inseefr.github.io/formation-bonnes-pratiques-git-R/slides/complete.html.

```
)
).to_df()
```

En R, il est possible d'écrire un code très proche quoiqu'un peu plus verbeux.

Un exemple d'utilisation des données diffusées par l'Insee avec DuckDB et R

```
library(duckdb)

url <- "https://www.data.gouv.fr/api/1/datasets/r/fdf5b008-054b-4065-a819-3eb3313f8be7"
con <- DBI::dbConnect(duckdb::duckdb(), ":memory:")
query <- glue::glue(
  "FROM read_parquet(\"{url}\") ",
  "SELECT COUNT(*) ",
  "WHERE COMMUNE = '06088' and CAST(AEMM AS int) > 2020"
)
dbGetQuery(con, query)
```

Il est néanmoins possible de directement utiliser la syntaxe très confortable de `dplyr` qui permet, par le biais du *package* `dbplyr`, de déléguer les calculs au moteur d'exécution de `duckdb`

Un exemple d'utilisation des données diffusées par l'Insee avec DuckDB et R via `dplyr`

```
library(dplyr)

url <- "https://www.data.gouv.fr/api/1/datasets/r/fdf5b008-054b-4065-a819-3eb3313f8be7"
con <- DBI::dbConnect(duckdb::duckdb(), ":memory:")
df <- tbl(con, glue::glue("read_parquet(\"{url}\")"))
df |>
  filter(COMMUNE == '06088' && AEMM > '2020') |>
  tally() |>
  collect()
```

Puisque les premières diffusions de Parquet en 2023 ont été appréciées par les utilisateurs des fichiers détails, ce format a été proposé dans la diffusion d'autres sources canonique de l'Insee. La Table 1 illustre les principales sources diffusées par l'Insee sur `data.gouv` en Parquet et le nombre de téléchargements de celles-ci. L'Insee n'est d'ailleurs pas la seule institution de la statistique publique à avoir adopté ce format de diffusion pour certaines sources volumineuses. Le service statistique ministériel du ministère de l'intérieur (le SSMSI) diffuse, depuis 2024, par exemple, les [bases statistiques de la délinquance](#) dans ce format de données.

L'acculturation progressive des utilisateurs comme des producteurs d'*open data* permet de prendre conscience de certaines bonnes pratiques à avoir dans la préparation du Parquet permettant de bénéficier des optimisations de ce fichier. Les principaux enjeux à avoir à l'esprit sont résumés par Mauviere (2024): définir des types de colonnes adéquats aux données représentées, trier le Parquet en fonction de variables qui servent fréquemment de filtres, privilégier une compression `zstd` à `snappy` (défaut), mettre à disposition ce parquet sur des serveurs autorisant les *range request*...

Table 1. – Principales sources de données diffusées par l’Insee sur data.gouv au format Parquet

Base	Page sur data.gouv	Téléchargements
Base Sirene des entreprises et de leurs établissements (SIREN, SIRET)	data.gouv.fr/datasets/base-sirene-des-entreprises-et-de-leurs-etablissements-siren-siret/	12 069 794
Géolocalisation des établissements du répertoire SIRENE-pour les études statistiques	data.gouv.fr/datasets/geolocalisation-des-etablissements-du-repertoire-sirene-pour-les-etudes-statistiques/	62 705
Bureaux de vote et adresses de leurs électeurs	data.gouv.fr/datasets/bureaux-de-vote-et-adresses-de-leurs-electeurs/	21 164
Recensement de la population - Fichiers détail : Individus localisés au canton-ou-ville	data.gouv.fr/datasets/recensement-de-la-population-fichiers-detail-individus-localises-au-canton-ou-ville/	4 481
Recensement de la population - Fichiers détail : Logements ordinaires	data.gouv.fr/datasets/recensement-de-la-population-fichiers-detail-logements-ordinaires/	3 921
Données sur la localisation et l’accès de la population aux équipements	data.gouv.fr/datasets/donnees-sur-la-localisation-et-lacces-de-la-population-aux-equipements/	2 524

Note: Les statistiques sur le nombre de téléchargement représentent ici tous les produits publiés sur la page en question, pas exclusivement les fichiers Parquet

4. Perspectives

Si ces premières expériences de diffusion de données au format Parquet ont permis d’illustrer l’apport de ce choix pour les *data scientists*, statisticiens et géomaticiens utilisant des données détaillées, ce n’est pas la seule communauté pouvant bénéficier de ce format. Les spécialistes de *data visualisation*, une communauté plus proche du développement web, utilisant plutôt le langage Javascript peuvent également bénéficier de ce format. Grâce au portage de DuckDB en *Web Assembly*⁴, il est possible de construire des *data visualisations* interactives capables de lire directement des blocs de données issus d’un fichier Parquet volumineux. Ces blocs peuvent être sélectionnés dynamiquement via une interface utilisateur, sans nécessiter de serveur intermédiaire comme c’est le cas pour les solutions type Shiny ou Steamlit. Lorsque le fichier Parquet est structuré de manière optimale, la lecture partielle devient extrêmement efficace, ce qui permet d’avoir une *data visualisation* très réactive même avoir des données de grande dimension⁵.

Le format Parquet n’a pas vocation à devenir le principal mode de diffusion des données en raison du profil des utilisateurs du site *insee.fr* dont les besoins portent majoritairement sur des données déjà agrégées et directement exploitables sans outil statistique. Il peut néanmoins constituer, dans de nombreux cas d’usage analytique, une alternative pertinente aux API pour l’accès aux données. Poussée à son extrême, la logique de consommation frugale d’un gros fichier permet d’envisager un modèle de diffusion de données faisant de fichiers Parquet de véritables API de données où, au lieu de définir des requêtes d’accès dans un JSON, celles-ci seraient définies à partir de requêtes SQL. C’est cette logique qu’adoptent certains acteurs centraux de l’écosystème Parquet ayant

⁴*WebAssembly* est une technologie complémentaire à JavaScript : elle permet d’exécuter dans le navigateur du code compilé à partir d’autres langages (comme C++, Rust ou Go). JavaScript reste le langage d’orchestration ; il charge, initialise et communique avec les modules *WebAssembly*, tandis que ces derniers exécutent les calculs lourds de manière plus efficace. Dans le contexte de la *data visualisation*, cela signifie qu’un moteur analytique comme DuckDB peut tourner directement dans le navigateur, sans serveur, tout en interagissant avec les bibliothèques JavaScript de visualisation (par exemple D3.js ou Vega-Lite).

⁵Pour en apprendre plus sur ce sujet, se reporter vers un atelier sur le sujet animé dans le cadre du réseau des data scientists de la statistique publique : <https://ssphub.github.io/ssphub-ateliers-parquet/>.

des besoins analytiques importants en proposant des lacs de données s'appuyant sur Parquet, en remplacement d'entrepôts de données reposant sur la logique des bases de données.

Bibliographie

Avouac, R. et Galiana, L. (2025) *Mise en production de projets data science*. Disponible sur: <https://doi.org/10.5281/zenodo.17486259>.

Dondon, A. et Lamarche, P. (2023) « Quels formats pour quelles données? », *Courrier des statistiques*, p. 86-103.

Mauviere, E. (2024) *Comment bien préparer son Parquet*. Disponible sur: <https://www.icem7.fr/outils/comment-bien-preparer-son-parquet/>.

Mühleisen, H. (2025) *The Lost Decade of Small Data?*. Disponible sur: <https://duckdb.org/2025/05/19/the-lost-decade-of-small-data>.

Tigani, J. (2023) *Big data is dead*. Disponible sur: <https://motherduck.com/blog/big-data-is-dead/>.